数据结构

1. 单调栈 2
    1.1. 应用 2
2. 单调队列 3
3. 并查集 4
    3.1. 带权并查集 4
    3.2. 种类并查集 5
4. 字典树 5
    4.1. 模板 5
5. 树状数组 7
    5.1. 优缺点 7
    5.2. 建树 7
    5.3. 应用 7
    5.4. 封装模板 9
    5.5. 高维树状数组 12
    5.6. PS 12
6. 线段树 12
    6.1. 经典模型 12
    6.2. 建树 14
    6.3. 单点修改 14
    6.4. 区间查询 15
    6.5. 区间修改 15
    6.6. 封装模板 15
    6.7. 动态开点线段树 21
    6.8. 势能线段树/吉司机线段树 23
    6.9. 线段树分裂 24
    6.10. 线段树合并 24
    6.11. 线段树优化建图 24
        6.11.1. PS 27
    6.12. 优化 30
    6.13. ps 31
    6.14. 经典模型 31
7. st 表 34
8. 启发式 35
    8.1. 启发式合并 35
    8.2. 树上启发式合并(dsu on tree) 37
9. 可持久化数据结构 39
    9.1. 可持久化线段树 39
        9.1.1. 单点修改,区间查询 42

9.1.2. 区间修改,区间查询 43
9.1.3. 应用 45

1. 单调栈

1.1. 应用

- 解决 NGE(Next Greater Element)问题
- 两元素间所有元素均(不)大/小于这两者问题

[模板]单调栈

```
int n, m;
int a[N], ans[N];
void solve()
{
    cin >> n;
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    stack<int> s;
    for (int i = 1; i <= n; i++)
    {
        while (s.size() && a[s.top()] < a[i])
        {
            ans[s.top()] = i;
            s.pop();
        }
        s.push(i);
    }
    for (int i = 1; i <= n; i++)
        cout << ans[i] << " ";
}
```

Patrik 音乐会的等待

```
int n, m;
int h[N];
void solve()
{
    cin >> n;
    for (int i = 1; i <= n; i++)
        cin >> h[i];
```

```
    stack<pii> s;// 这里pair的第二个成员表示相同元素的数量
    ll ans = 0;
    for (int i = 1; i <= n; i++)
    {
        int num = 0;
        while (s.size() && s.top().x <= h[i])
        {
            if (s.top().x == h[i])
                num = s.top().y;
            ans += s.top().y;
            s.pop();
        }
        if (s.size())
            ans++;
        s.push({h[i], num + 1});
    }
    cout << ans << endl;
}
```

2. 单调队列

滑动窗口 / 【模板】单调队列

给出一个长度为 n 的数组,输出每 k 个连续的数中的最大值和最小值。

```
int n, k;
int a[N], q[N];
void solve()
{
    cin >> n >> k;
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    int l = 1, r = 0;
    for (int i = 1; i <= n; i++)
    {
        while (l <= r && a[i] <= a[q[r]])
            r--;
        q[++r] = i;
        while (q[l] <= i - k)
            l++;
```

```
        if (i >= k)
            cout << a[q[l]] << " ";
    }
    cout << endl;
    l = 1, r = 0;
    for (int i = 1; i <= n; i++)
    {
        while (l <= r && a[i] >= a[q[r]])
            r--;
        q[++r] = i;
        while (q[l] <= i - k)
            l++;
        if (i >= k)
            cout << a[q[l]] << " ";
    }
    cout << endl;
}
```

3. 并查集

3.1. 带权并查集

The Third Letter

已知若干对士兵的相对位置,询问是否合法

```
void solve()
{
    int n, m;
    cin >> n >> m;
    vector<ll> p(n + 1), d(n + 1);
    for (int i = 1; i <= n; i++)
        p[i] = i, d[i] = 0;
    auto find = [&](const auto &find, ll x) -> ll
    {
        if (x != p[x])
        {
            ll res = find(find, p[x]);
            d[x] += d[p[x]];
            p[x] = res;
        }
```

```
        return p[x];
    };
    bool ok = true;
    for (int i = 1; i <= m; i++)
    {
        ll u, v, w;
        cin >> u >> v >> w;
        ll fa = find(find, u), fb = find(find, v);
        if (fa != fb)
        {
            p[fa] = fb;
            d[fa] = d[v] - d[u] + w;
        }
        else if (ok && d[u] - w != d[v])
        {
            ok = false;
            cout << "NO" << endl;
        }
    }
    if (ok)
        cout << "YES" << endl;
}
```

3.2. 种类并查集

4. 字典树

4.1. 模板

```
struct Trie
{
    int root, tot, nex[N * M][2];
    ll val[N * M]; // N 输入的字符串的长度和 ,M 字符集
    int newnode()
    {
        memset(nex[tot], 0, sizeof nex[tot]);
        val[tot] = 0;
        return tot++;
    }
```

```
    void init()
    {
        memset(nex[0], 0, sizeof nex[0]);
        val[0] = 0;
        tot = 1;
        root = newnode();
    }
    void upd(ll x, ll v)
    {
        int id = root;
        for (int i = 30; ~i; i--)
        {

            int t = (x >> i) & 1;
            if (!nex[id][t])
                nex[id][t] = newnode();
            id = nex[id][t];
            val[id] = (val[id] + v) % mod;
        }
    }
    ll ask(ll x, ll k)
    {
        int id = root;
        ll res = 0;
        for (int i = 30; ~i; i--)
        {
            int t = (x >> i) & 1;
            if (((k >> i) & 1) == 1)
            {
                if (nex[id][t])
                    res = (res + val[nex[id][t]]) % mod;
                id = nex[id][t ^ 1];
            }
            else
                id = nex[id][t];
        }
        res = (res + val[id]) % mod;
        return res;
    }
```

```
} tr;
```

5. 树状数组

5.1. 优缺点

功能是线段树的子集,但是代码与常数相对线段树都较小,优点是代码量少.

5.2. 建树

- 可以通过单点修改来建树,但是这样是$O(\log n)$的.
- 每个 c[i]分管的范围是[x-lowbit(x),x]

对于每个右端点 R,其树状数组的长度是 lowbit(R),所以可以通过前缀和实现$O(n)$建树

```
void build()
{
    for (int i = 1; i <= n; ++i)
    {
        // c[i] = [x-lowbit(x)+1,x]
        // s 是前缀和数组
        c[i] = s[i] - s[i - lowbit(i)];
    }
}
```

- 暴力建树之所以会超时,是因为多次修改了重复节点,但是注意到每个 c[i]的值都是由更小的 c[i]得到的,所以可以通过从小到大的顺序,由子节点更新父节点,这样每个节点只会更新一次,这样也是 O(n)的了.

```
void build()
{
    for (int i = 1; i <= n; ++i)
    {
        c[i] += a[i];
        int fa = i + lowbit(i);
        if (fa <= n)
            c[fa] += c[i];
    }
}
```

5.3. 应用

- 单点修改,区间查询

因为树状数组的查询是前缀和,所以只需要输出 query (r) - query (l-1)

```
bit.add(pos, x);
cout << bit.query(r) - bit.query(l - 1) << endl;
```

- 单点查询,区间修改

树状数组维护差分数组,每次区间修改视为普通的差分修改,单点查询时直接输出 query(pos)

```
//区间修改
bit.sum += x;//sum 相当于 a[1]
bit.add(i, x);
bit.add(i + 1, -x);
//单点查询
cout << bit.query(pos) << endl;
```

- 区间修改,区间查询

用两个树状数组分别维护数组的差分$c[i]$以及$c[i]*i$

```
void add(int l, int r, ll d)
{
    bit1.add(l, d);
    bit1.add(r + 1, -d);
    bit2.add(l, l * d);
    bit2.add(r + 1, (r + 1) * -d);
}
ll query(int l, int r)
{
    return (r + 1) * bit1.query(r) - bit2.query(r) - (l*bit1.query(l - 1) - bit2.query(l - 1));
}
```

- 树状数组二分

```
int query(ll s)
{
    int pos = 0;
    for (int j = 18; j >= 0; j--)
        if (pos + (1 << j) <= n && c[pos + (1 << j)] <= s)
        {
            pos += (1 << j);
```

```
            s -= c[pos];
        }
    return pos;
}
```

- 求逆序对

```
for (int i = 1; i <= n; i++)
{
    cin >> a[i];
    a[i] = n + 1 - a[i];
    ans += query(a[i]);
    modify(a[i], 1);
}
```

5.4. 封装模板

单点加,区间和

```
struct BIT
{
    int n;
    vector<ll> a;
    BIT(int _n) : n(_n), a(n + 1) {}
    int lb(int x) { return x & -x; }
    void build(int n, vector<int> &s)
    {
        for (int i = 1; i <= n; i++)
        {
            a[i] += s[i];
            int fa = i + lb(i);
            if (fa <= n)
                a[fa] += a[i];
        }
    }
    void add(int x, ll y)
    {
        for (; x <= n; x += lb(x))
            a[x] += y;
    }
    ll query(int x)
```

```
    {
        ll res = 0;
        for (; x; x ^= lb(x))
            res += a[x];
        return res;
    }
    int search(ll s)//第一个小于等于 s 的位置
    {
        int pos = 0;
        for (int j = 20; j >= 0; j--)
            if (pos + (1 << j) <= n && a[pos + (1 << j)] <= s)
            {
                pos += (1 << j);
                s -= a[pos];
            }
        return pos;
    }
};
```

区间加,单点查询

用树状数组维护差分数组

```
struct BIT {
    int n;
    vector<ll> a;
    BIT(int _n) : n(_n), a(n + 10) {}
    int lb(int &x) { return x & -x; }
    void add(int pos, ll x) {
        for (; pos <= n; pos += lb(pos))
            a[pos] += x;
    }
    void add(int l, int r, ll x) {
        add(l, x), add(r + 1, -x);
    }
    ll query(int x) {
        ll res = 0;
        for (; x > 0; x ^= lb(x))
            res += a[x];
        return res;
```

```
    }
};
```

区间加,区间和

区间和:

将区间询问转化为两次前缀和询问

树状数组查询前缀和,实际上是对差分数组做两次前缀和

$$
\begin{aligned}
[1,r] &= \sum_{i=1}^{r}\sum_{j=1}^{i} d_j \\
&= \sum_{i=1}^{r} d_i(r-i+1) \\
&= \sum_{i=1}^{r} d_i(r+1) - \sum_{i=1}^{r} d \times i
\end{aligned}
$$

用两个树状数组分别维护即可实现询问区间和

区间加:

在差分数组上实现区间加,实际上只需要操作d_l, d_{r+1}

对维护d_i的树状数组,$l+v, r+-v$

对维护$d_i \times i$的树状数组,$l+v\times l, r+-v\times(r+1)$

```
struct BIT {
    int n;
    vector<ll> t1, t2;
    int lb(int x) { return x & (-x); }
    BIT(int _n) : n(_n), t1(n + 10), t2(n + 10) {}
    void add(int k, ll v) {
        int v1 = k * v;
        for (; k <= n; k += lb(k)) {
            t1[k] += v, t2[k] += v1;
            // 注意不能写成 t2[k] += k * v, 因为 k 的值已经不是原数组的下标了
        }
    }
    ll qry(vector<ll> &t, int k) {
        ll res = 0;
        for (; k > 0; k ^= lb(k))
```

```
            res += t[k];
        return res;
    }
    void add(int l, int r, ll v) {
        add(l, v), add(r + 1, -v); // 将区间加差分为两个前缀加
    }
    ll qry(int l, int r) {
        return (r + 1ll) * qry(t1, r) - 1ll * l * qry(t1, l - 1) - (qry(t2, r) - qry(t2, l - 1));
    }
};
```

5.5. 高维树状数组

```
void modify(int x, int y, ll s)
{
    for (int p = x; p <= n; p += p & (-p))
        for (int q = y; q <= m; q += q & (-q))
            c[p][q] += s;
}
ll query(int x, int y)
{
    ll s = 0;
    for (int p = x; p; p -= p & (-p))
        for (int q = y; q; q -= q & (-q))
            s += c[p][q];
    return s;
}
```

5.6. PS

树状数组维护信息的下标只能从 1 开始!否则会造成死循环。

6. 线段树

6.1. 经典模型

1.二维数点

加入点要放在询问前面

坐标要加一防止出现 0

https://www.luogu.com.cn/problem/P2163

```
struct BIT
{
    int n;
    vector<int> a;
    BIT(int n) : n(n), a(n + 10) {}
    int lb(int x) { return x & -x; }
    void add(int x, ll s)
    {
        for (; x <= n; x += lb(x))
            a[x] += s;
    }
    ll query(int x)
    {
        ll res = 0;
        for (; x; x ^= lb(x))
            res += a[x];
        return res;
    }
};
void solve()
{
    int n, m;
    cin >> n >> m;
    BIT bit(1e7 + 10);
    vector<pii> p(n);
    vector<int> ans(m + 1, 0);
    vector<array<int, 5>> opt;
    for (int i = 1; i <= n; i++)
    {
        int x, y;
        cin >> x >> y;
        x++, y++;
        opt.push_back({x, y, 0, 0, 0});
    }
    for (int i = 1; i <= m; i++)
    {
        int x1, y1, x2, y2;
        cin >> x1 >> y1 >> x2 >> y2;
        x1++, y1++, x2++, y2++;
```

```
        opt.push_back({x1 - 1, y1 - 1, 1, i, 1}); // x,y,操作类型,答案对应位置,+/-
        opt.push_back({x2, y1 - 1, 1, i, -1});
        opt.push_back({x1 - 1, y2, 1, i, -1});
        opt.push_back({x2, y2, 1, i, 1});
    }
    sort(opt.begin(), opt.end());
    for (auto &[x, y, v, p, t] : opt)
    {
        if (!v) // 加点
        {
            bit.add(y, 1);
        }
        else
        {
            ll res = bit.query(y);
            ans[p] += t * res;
        }
    }
    for (int i = 1; i <= m; i++)
        cout << ans[i] << endl;
}
```

6.2. 建树

```
void build(int id, int l, int r)
{
    if (l == r)
        seg[id].val = 0;
    else
    {
        int mid = l + r >> 1;
        build(id * 2, l, mid);
        build(id * 2 + 1, mid + 1, r);
        update(id);
    }
}
```

6.3. 单点修改

```cpp
// 节点为 id,对应区间为[l,r],a[pos]->val
void change(int id, int l, int r, int pos, int val)
{
    if (l == r) // 叶子节点
        seg[id].val = val;
    else
    {
        int mid = l + r >> 1;
        if (pos <= mid)
            change(id * 2, l, mid, pos, val);
        else
            change(id * 2 + 1, mid + 1, r, pos, val);
        update(id);
    }
}
```

6.4. 区间查询

```cpp
ll query(int id, int l, int r, int ql, int qr)
{
    if (l == ql && r == qr)
        return seg[id].val;
    int mid = l + r >> 1;
    if (qr <= mid)
        return query(id * 2, l, mid, ql, qr);
    else if (ql > mid)
        return query(id * 2 + 1, mid + 1, r, ql, qr);
    else
        return query(id * 2, l, mid, ql, mid) + query(id * 2 + 1, mid + 1, r, mid + 1, qr);
}
```

6.5. 区间修改

在修改时设置懒标记,查询时将标记下放.

```
```

6.6. 封装模板

单点修改,区间查询

```cpp
struct node
{
    ll pos, col;
};
struct segtree
{
    int n;
    vector<node> a;
    segtree(int _n) : n(_n * 4 + 10), a(n + 1) {}
    void update(int id)
    {
        a[id].pos = min(a[id * 2].pos, a[id * 2 + 1].pos);
        if (a[id * 2].pos < a[id * 2 + 1].pos)
            a[id].col = a[id * 2].col;
        else
            a[id].col = a[id * 2 + 1].col;
    }
    void build(int id, int l, int r, vector<int> &arr)
    {
        if (l == r)
            a[id].pos = arr[l], a[id].col = l;
        else
        {
            int mid = l + r >> 1;
            build(id * 2, l, mid, arr);
            build(id * 2 + 1, mid + 1, r, arr);
            update(id);
        }
    }
    void change(int id, int l, int r, int pos, int t)
    {
        if (l == r) // 叶子节点
            a[id].pos = t;
        else
        {
            int mid = l + r >> 1;
            if (pos <= mid)
                change(id * 2, l, mid, pos, t);
            else
```

```
                change(id * 2 + 1, mid + 1, r, pos, t);
            update(id);
        }
    }
    pii query(int id, int l, int r, int ql, int qr)
    {
        if (l == ql && r == qr)
            return {a[id].pos, a[id].col};
        int mid = l + r >> 1;
        if (qr <= mid)
            return query(id * 2, l, mid, ql, qr);
        else if (ql > mid)
            return query(id * 2 + 1, mid + 1, r, ql, qr);
        else
        {
            auto t1 = query(id * 2, l, mid, ql, mid);
            auto t2 = query(id * 2 + 1, mid + 1, r, mid + 1, qr);
            if (t1.x < t2.x)
                return t1;
            return t2;
        }
    }
};
```

区间加,区间和

```
struct segtree
{
    struct node
    {
        ll t, val, sz;
    };
    int n;
    vector<node> a;
    segtree(int _n) : n(_n * 4 + 10), a(n + 1) {}
    void update(int id) { a[id].val = a[id * 2].val + a[id * 2 + 1].val; }
    void settag(int id, int t)
    {
        a[id].val = a[id].val + t * (a[id].sz);
```

```
        a[id].t = a[id].t + t;
    }
    void pushdown(int id)
    {
        if (a[id].t)
        {
            settag(id * 2, a[id].t);
            settag(id * 2 + 1, a[id].t);
            a[id].t = 0;
        }
    }
    void build(int id, int l, int r, vector<int> &arr)
    {
        a[id].t = 0;
        a[id].sz = r - l + 1;
        if (l == r)
            a[id].val = arr[l];
        else
        {
            int mid = l + r >> 1;
            build(id * 2, l, mid, arr);
            build(id * 2 + 1, mid + 1, r, arr);
            update(id);
        }
    }
    void modify(int id, int l, int r, int ql, int qr, int t)
    {
        if (r < ql || l > qr)
            return;
        if (l >= ql && r <= qr)
        {
            settag(id, t);
            return;
        }
        int mid = l + r >> 1;
        pushdown(id);
        if (ql <= mid)
            modify(id * 2, l, mid, ql, qr, t);
        if (qr > mid)
```

```
            modify(id * 2 + 1, mid + 1, r, ql, qr, t);
        update(id);
    }
    ll query(int id, int l, int r, int ql, int qr)
    {
        if (r < ql || l > qr)
            return 0ll;
        if (l >= ql && r <= qr)
            return a[id].val;
        int mid = l + r >> 1;
        pushdown(id);
        ll res = 0;
        if (ql <= mid)
            res += query(id * 2, l, mid, ql, qr);
        if (qr > mid)
            res += query(id * 2 + 1, mid + 1, r, ql, qr);
        return res;
    }
};
```

区间加,区间赋值,区间乘,区间查询

```
struct tag
{
    ll mul, add;
    tag operator+(const tag &t) const
    {
        return {mul * t.mul % mod, (add * t.mul + t.add) % mod};
    }
};
struct node
{
    tag t;
    ll val, sz;
};
struct segtree
{
    int n;
    vector<node> a;
```

```
    segtree(int _n) : n(_n * 4 + 10), a(n + 1) {}
    void update(int id) { a[id].val = (a[id * 2].val + a[id * 2 + 1].val) % mod; }
    void settag(int id, tag t)
    {
        a[id].val = (a[id].val * t.mul + a[id].sz * t.add) % mod;
        a[id].t = a[id].t + t;
    }
    void pushdown(int id)
    {
        if (a[id].t.mul != 1 || a[id].t.add) // 标记非空
        {
            settag(id * 2, a[id].t);
            settag(id * 2 + 1, a[id].t);
            a[id].t.add = 0, a[id].t.mul = 1;
        }
    }
    void build(int id, int l, int r, vector<int> &arr)
    {
        a[id].t = {1, 0};
        a[id].sz = r - l + 1;
        if (l == r)
            a[id].val = {arr[l]};
        else
        {
            int mid = l + r >> 1;
            build(id * 2, l, mid, arr);
            build(id * 2 + 1, mid + 1, r, arr);
            update(id);
        }
    }
    void modify(int id, int l, int r, int ql, int qr, tag t)
    {
        if (l == ql && r == qr)
        {
            settag(id, t);
            return;
        }
        int mid = l + r >> 1;
```

```
        pushdown(id);
        if (qr <= mid)
            modify(id * 2, l, mid, ql, qr, t);
        else if (ql > mid)
            modify(id * 2 + 1, mid + 1, r, ql, qr, t);
        else
        {
            modify(id * 2, l, mid, ql, mid, t);
            modify(id * 2 + 1, mid + 1, r, mid + 1, qr, t);
        }
        update(id);
    }
    ll query(int id, int l, int r, int ql, int qr)
    {
        if (l == ql && r == qr)
            return a[id].val;
        int mid = l + r >> 1;
        pushdown(id);
        if (qr <= mid)
            return query(id * 2, l, mid, ql, qr);
        else if (ql > mid)
            return query(id * 2 + 1, mid + 1, r, ql, qr);
        else
        {
            return (query(id * 2, l, mid, ql, mid) + query(id * 2 + 1, mid + 1, r, mid + 1, qr)) % mod;
        }
    }
};
```

6.7. 动态开点线段树

结点只有在有需要的时候才被创建

```
#define ls(x) (a[x].l)
#define rs(x) (a[x].r)
#define sum(x) a[x].sum

struct node
{
```

```
    ll sum, l, r;
};
struct segtree
{
    int n;
    vector<node> a;
    segtree(int _n) : n(0) { a.reserve(_n); }
    void add(ll &id, int l, int r, int pos, int t) // add
    {
        if (!id)
            id = ++n;
        sum(id) += t;
        if (l == r)
            return;
        int mid = l + r >> 1;
        if (pos <= mid)
            add(ls(id), l, mid, pos, t);
        else
            add(rs(id), mid + 1, r, pos, t);
    }
    void change(ll &id, int l, int r, int pos, int t) // change
    {
        if (!id)
            id = ++n;
        if (l == r)
        {
            sum(id) = t;
            return;
        }
        int mid = l + r >> 1;
        if (pos <= mid)
            change(ls(id), l, mid, pos, t);
        else
            change(rs(id), mid + 1, r, pos, t);
        sum(id) = sum(ls(id)) + sum(rs(id));
    }
    ll query(ll id, int l, int r, int ql, int qr)
    {
        if (!id || r < ql || l > qr)
```

```
            return 0;
        if (l >= ql && r <= qr)
            return sum(id);
        int mid = l + r >> 1;
        return query(ls(id), l, mid, ql, qr) + query(rs(id), mid + 1, r, ql, qr);
    }
};
```

6.8. 势能线段树/吉司机线段树

一般线段树的区间修改是基于懒标记的,但是有些问题,比如区间开根号,区间取模等,这些操作是基于叶子节点的值的,无法通过懒标记修改,只能暴力到叶子节点

这个时候就需要势能线段树,可以简单理解为,总操作数不多,所以暴力操作也不会超时

```
void modify(int id, int l, int r, int ql, int qr, ll t)
{
    if (l >= ql && r <= qr && a[id].mx < t)
        return;
    if (r < ql || l > qr)
        return;
    if (l == r)
    {
        ll res = a[id].val % t;
        a[id].val = a[id].mx = res;
        return;
    }
    int mid = l + r >> 1;
    if (ql <= mid)
        modify(id * 2, l, mid, ql, qr, t);
    if (qr > mid)
        modify(id * 2 + 1, mid + 1, r, ql, qr, t);
    update(id);
}
```

6.9. 线段树分裂

6.10. 线段树合并

6.11. 线段树优化建图

https://codeforces.com/contest/786/problem/B

有 n 个点,q 个询问,每次询问给出一个操作。

操作 1: 1 u v w,从 u 向 v 连一条权值为 w 的有向边

操作 2: 2 u l r w,从 u 向区间[l,r]的所有点连一条权值为 w 的有向边

操作 3: 3 u l r w,从区间[l,r]的所有点连一条权值为 w 的有向边

求某点到其他点的最短路

```
#include <bits/stdc++.h>
#define endl '\n'
#define x first
#define y second
#define sum(x) (a[x].sum)
#define tag(x) (a[x].t)
#define ls(x) (a[x].l)
#define rs(x) (a[x].r)
using namespace std;
typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef double db;
const ll mod = 998244353;
const int N = 1e6 + 100, M = 5e5 + 10;
vector<pll> e[N];
int opt;
ll a[N], dist[N];
bool vis[N];
void build(int id, int l, int r)
{
    if (l == r)
    {
        a[l] = id;
        return;
```

```
    }
    int mid = l + r >> 1;
    e[id].push_back({id * 2, 0});
    e[id].push_back({id * 2 + 1, 0});
    // 反向边,用虚点建边
    e[id * 2 + M].push_back({id + M, 0});
    e[id * 2 + 1 + M].push_back({id + M, 0});
    build(id * 2, l, mid), build(id * 2 + 1, mid + 1, r);
}
void modify(int id, int l, int r, int ql, int qr, int v, int w)
{
    if (r < ql || l > qr)
        return;
    if (l >= ql && r <= qr)
    {
        if (opt == 2)
            e[v + M].push_back({id, w});
        else
            e[id + M].push_back({v, w});
        return;
    }
    int mid = l + r >> 1;
    modify(id * 2, l, mid, ql, qr, v, w);
    modify(id * 2 + 1, mid + 1, r, ql, qr, v, w);
}

void solve()
{
    int n, m, s;
    cin >> n >> m >> s;
    build(1, 1, n);
    for (int i = 1; i <= n; i++)
        e[a[i]].push_back({a[i] + M, 0}), e[a[i] + M].push_back({a[i], 0});
    for (int i = 1; i <= m; i++)
    {
        cin >> opt;
        if (opt == 1)
        {
```

```
            ll u, v, w;
            cin >> u >> v >> w;
            e[a[u]].push_back({a[v], w});
        }
        else
        {
            ll x, l, r, w;
            cin >> x >> l >> r >> w;
            modify(1, 1, n, l, r, a[x], w);
        }
    }
    auto dij = [&](int s) -> void
    {
        for (int i = 1; i <= 1e6; i++)
            dist[i] = 1e18;
        dist[s] = 0;
        priority_queue<pll, vector<pll>, greater<pll>> pq;
        pq.push({0, s});
        while (pq.size())
        {
            auto [d, t] = pq.top();
            pq.pop();
            if (vis[t])
                continue;
            vis[t] = 1;
            for (auto [to, w] : e[t])
            {
                if (dist[to] > dist[t] + w)
                    dist[to] = dist[t] + w, pq.push({dist[to], to});
            }
        }
    };
    dij(a[s] + M);
    for (int i = 1; i <= n; i++)
        cout << (dist[a[i]] == 1e18 ? -1 : dist[a[i]]) << " ";
}
int main()
{
```

```
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    int _ = 1;
    // cin >> _;
    while (_--)
        solve();
    return 0;
}
```

6.11.1. PS

假如题目要求在区间$[l,r]$与区间[x,y]之间连边,可以新开一个中间点t,即

$$[l,r] \leftrightarrow t \leftrightarrow [x,y]$$

这样就又转化成了点对区间连边

https://codeforces.com/gym/105174/problem/J

```
#include <bits/stdc++.h>
#define endl '\n'
#define x first
#define y second
#define debug cout << "debug-------" << endl
using namespace std;
using ll = long long;
using pii = pair<int, int>;
using db = long double;
const int N = 1e6 + 100, M = 5e5;
vector<int> e[N], E[N];
int leaf[N], dfn[N], low[N], bel[N], deg[N], L[N], R[N];
int in[N], out[N];
bool vis[N];
int mx, idx, ans; // 最大点数,dfn,scc的数量
vector<int> stk;
void build(int id, int l, int r) {

    if (l == r) {
        in[id] = ++mx;
        leaf[l] = mx;
        out[id] = ++mx;
        mx = max(mx, id);
```

```
        e[out[id]].push_back(in[id]);
        e[in[id]].push_back(out[id]);
        return;
    }
    in[id] = ++mx;
    out[id] = ++mx;
    int mid = (l + r) >> 1;
    build(id * 2, l, mid), build(id * 2 + 1, mid + 1, r);
    e[out[id]].push_back(out[id * 2]);
    e[out[id]].push_back(out[id * 2 + 1]);
    e[in[id * 2]].push_back(in[id]);
    e[in[id * 2 + 1]].push_back(in[id]);
}
void modify(int id, int l, int r, int ql, int qr, int v) {
    if (r < ql || l > qr)
        return;
    if (l >= ql && r <= qr) {
        if (v)
            e[in[id]].push_back(mx);
        else
            e[mx].push_back(out[id]);
        return;
    }
    int mid = (l + r) >> 1;
    modify(id * 2, l, mid, ql, qr, v);
    modify(id * 2 + 1, mid + 1, r, ql, qr, v);
}
void tarjan(int u) {
    dfn[u] = low[u] = ++idx;
    stk.push_back(u);
    vis[u] = 1;
    for (auto &to : e[u]) {
        if (!dfn[to]) {
            tarjan(to);
            low[u] = min(low[u], low[to]);
        } else if (vis[to])
            low[u] = min(low[u], dfn[to]);
    }
    if (dfn[u] == low[u]) {
```

```
        ans++;
        while (1) {
            auto t = stk.back();
            stk.pop_back();
            vis[t] = 0;
            bel[t] = ans;
            if (t == u)
                break;
        }
    }
}
void solve() {
    int n, m;
    mx = 0;
    cin >> n >> m;
    build(1, 1, n);
    for (int i = 1; i <= m; i++) {
        int l, r, x, y;
        cin >> l >> r >> x >> y;
        mx++;
        modify(1, 1, n, l, r, 1); //[l,r]->mx
        modify(1, 1, n, x, y, 0); // mx->[x,y]
    }

    for (int i = 1; i <= mx; i++) {
        if (!dfn[i])
            tarjan(i);
    }
    for (int i = 1; i <= mx; i++)
        L[i] = 1e9;
    for (int i = n; i >= 1; i--)
        L[bel[leaf[i]]] = i;
    for (int i = 1; i <= n; i++)
        R[bel[leaf[i]]] = i;

    for (int i = 1; i <= mx; i++)
        for (auto &to : e[i])
            if (bel[to] != bel[i])
                E[bel[to]].push_back(bel[i]), deg[bel[i]]++;
```

```
    queue<int> q;
    for (int i = 1; i <= ans; i++)
        if (!deg[i])
            q.push(i);
    while (q.size()) {
        auto t = q.front();
        q.pop();
        for (auto &to : E[t]) {
            if (--deg[to] == 0)
                q.push(to);
            L[to] = min(L[to], L[t]), R[to] = max(R[to], R[t]);
        }
    }
    for (int i = 1; i <= n; i++)
        cout << L[bel[leaf[i]]] << " " << R[bel[leaf[i]]] << endl;
    for (int i = 1; i <= mx; i++) {
        e[i].clear(), E[i].clear();
        dfn[i] = low[i] = bel[i] = deg[i] = 0;
        vis[i] = 0;
        L[i] = 1e9, R[i] = 0;
    }
    idx = ans = 0;
}
int main() {
#ifdef ix3ea
    freopen("in.txt", "r", stdin);
    freopen("out.txt", "w", stdout);
#endif
    cout << fixed << setprecision(12);
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    int _ = 1;
    cin >> _;
    while (_--)
        solve();
    return 0;
}
```

6.12. 优化

1. 在叶子节点处无需下放懒惰标记,所以懒标记可以不下传到叶子节点。

2.标记永久化:如果确定懒惰标记不会在中途被加到溢出（即超过了该类型数据所能表示的最大范围）,那么就可以将标记永久化。标记永久化可以避免下传懒惰标记,只需在进行询问时把标记的影响加到答案当中,从而降低程序常数。具体如何处理与题目特性相关,需结合题目来写。这也是树套树和可持久化数据结构中会用到的一种技巧

6.13. ps

- 如果数组的大小在$2^{\{n\}}+1-2^{\{n+1\}}$之间,那么线段树的大小可能会达到$2^{\{n+2\}}$,最坏情况下是 4 倍的大小,所以要开 4 倍
- 用线段树解决问题需要满足能高效合并两个区间的信息
- 线段树维护信息的下标只能从 1 开始!
- 注意是否会出现$l>r$的情况,这种情况会无限递归
- 线段树需要支持能够高效合并两个区间
- 用 vector 实现线段树时,因为 vector 的存储是自动管理的,按需扩张收缩,vector 通常占用多于静态数组的空间,因为要分配更多内存以管理将来的增长。

可以使用 reserve 预先增大 vector 的容量。

6.14. 经典模型

带修区间最大子段和

https://www.luogu.com.cn/problem/SP1716

```
#include <bits/stdc++.h>
#define endl '\n'
#define x first
#define y second
using namespace std;
typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef double db;
const ll mod = 998244353;
const int N = 5e5 + 10, M = 25;
random_device rd;        // 将用于为随机数引擎获得种子
mt19937_64 gen(time(0)); // 以播种标准 mersenne_twister_engine
struct node
{
    ll ls, rs, ms, s;
};
```

```
struct segtree
{
    int n;
    vector<node> a;
    segtree(int _n) : n(_n * 4 + 10), a(n + 1) {}
    void update(int id)
    {
        a[id].ls = max(a[id * 2].s + a[id * 2 + 1].ls, a[id * 2].ls);
        a[id].rs = max(a[id * 2 + 1].s + a[id * 2].rs, a[id * 2 + 1].rs);
        a[id].s = a[id * 2].s + a[id * 2 + 1].s;
        a[id].ms = max({a[id * 2].ms, a[id * 2 + 1].ms, a[id * 2].rs + a[id * 2 + 1].ls});
    }
    void build(int id, int l, int r, vector<int> &arr)
    {
        if (l == r)
            a[id].s = a[id].ms = a[id].ls = a[id].rs = arr[l];
        else
        {
            int mid = l + r >> 1;
            build(id * 2, l, mid, arr);
            build(id * 2 + 1, mid + 1, r, arr);
            update(id);
        }
    }
    void change(int id, int l, int r, int pos, int t)
    {
        if (l == r) // 叶子节点
            a[id].s = a[id].ms = a[id].ls = a[id].rs = t;
        else
        {
            int mid = l + r >> 1;
            if (pos <= mid)
                change(id * 2, l, mid, pos, t);
            else
                change(id * 2 + 1, mid + 1, r, pos, t);
            update(id);
```

```
        }
    }
    node query(int id, int l, int r, int ql, int qr)
    {
        if (l == ql && r == qr)
            return a[id];
        int mid = l + r >> 1;
        if (qr <= mid)
            return query(id * 2, l, mid, ql, qr);
        else if (ql > mid)
            return query(id * 2 + 1, mid + 1, r, ql, qr);
        else
        {
            auto t1 = query(id * 2, l, mid, ql, mid);
            auto t2 = query(id * 2 + 1, mid + 1, r, mid + 1, qr);
            node t;
            t.s = t1.s + t2.s;
            t.ls = max(t1.s + t2.ls, t1.ls);
            t.rs = max(t2.s + t1.rs, t2.rs);
            t.ms = max({t1.ms, t2.ms, t1.rs + t2.ls});
            return t;
        }
    }
};
void solve()
{
    int n, q;
    cin >> n;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    segtree seg(n);
    seg.build(1, 1, n, a);
    cin >> q;
    int opt, l, r;
    for (int i = 1; i <= q; i++)
    {
        cin >> opt >> l >> r;
        if (opt == 0)
```

```
                seg.change(1, 1, n, l, r);
            else
                cout << seg.query(1, 1, n, l, r).ms << endl;
        }
    }
    int main()
    {
        // cout << setprecision(5);
        ios::sync_with_stdio(false);
        cin.tie(0), cout.tie(0);
        int _ = 1;
        // cin >> _;
        while (_--)
            solve();
        return 0;
    }
```

7. st 表

```
struct ST
{
    int n, q;
    vector<int> lg;
    vector<vector<ll>> f;
    ST(int _n) : n(_n), lg(n + 3, 0), f(25, vector<ll>(n + 1, 0)) {}
    void init(vector<int> &a)
    {
        lg[1] = 0;
        for (int i = 2; i <= n; i++)
            lg[i] = lg[i / 2] + 1;
        for (int i = 1; i <= n; i++)
            f[0][i] = a[i];
        for (int j = 1; j <= 20; j++)
            for (int i = 1; i + (1 << j) - 1 <= n; i++)
                f[j][i] = max(f[j - 1][i], f[j - 1][i + (1 << (j - 1))]);
    }
    int query(int l, int r)
```

```
    {
        int len = lg[r - l + 1];
        // 或者写成__lg(r - l + 1);
        return max(f[len][l], f[len][r - (1 << len) + 1]);
    }
};
```

```
template<class T>
struct SparseTable {
    vector<T> v;
    vector<vector<T>> f;
    function<T(T, T)> merge;
    SparseTable(vector<T> v, function<T(T, T)> merge) : v(v), merge(merge) {
        int n = v.size() - 1, m = __lg(n);
        f = vector<vector<T>>(m + 1, vector<T>(n + 1));
        for (int j = 0; j <= m; j++) {
            for (int i = 1; i + (1 << j) - 1 <= n; i++) {
                if (j == 0) {
                    f[j][i] = v[i];
                } else {
                    f[j][i] = merge(f[j - 1][i], f[j - 1][i + (1 << j - 1)]);
                }
            }
        }
    }
    T ask(int l, int r) {
        int k = __lg(r - l + 1);
        return merge(f[k][l], f[k][r - (1 << k) + 1]);
    }
};
```

8. 启发式

8.1. 启发式合并

https://www.luogu.com.cn/problem/P3201

n 个布丁摆成一行,每个布丁都有一个颜色a_i,进行 m 次操作。

操作共有 2 种:

将颜色为 x 的布丁全部变成颜色 y 的布丁

询问当前一共有多少段颜色

```
#include <bits/stdc++.h>
#define endl '\n'
#define x first
#define y second
#define stop cout << "?" << '\n'
using namespace std;
typedef long long ll;
typedef pair<int, int> pii;
typedef pair<int, int> PII;
const int N = 1e6 + 100, M = 1e2;
const ll mod = 1e9 + 7;
const double pi = acos(-1);
int n, m;
vector<int> pos[N];
int a[N];
void solve()
{
    int ans = 0;
    cin >> n >> m;
    for (int i = 1; i <= n; i++)
    {
        cin >> a[i];
        pos[a[i]].push_back(i);
    }
    a[0]=a[n+1]=0;
    for (int i = 1; i <= n; i++)
        ans += (a[i] != a[i - 1]);
    while (m--)
    {
        int op, x, y;
        cin >> op;
        if (op == 2)
            cout << ans << endl;
        else
        {
            cin >> x >> y;
```

```
            if(x==y)
                continue;
            if (pos[x].size() > pos[y].size())
                swap(pos[x], pos[y]);
            if(pos[x].empty())
                continue;
            auto modify = [&](int i, int col){
                ans -= (a[i] != a[i - 1]) + (a[i] != a[i + 1]);
                a[i] = col;
                ans += (a[i] != a[i - 1]) + (a[i] != a[i + 1]);
            };
            int col=a[pos[y][0]];
            for (auto i : pos[x])
            {
                modify(i,col);
                pos[y].push_back(i);
            }
            pos[x].clear();
        }
    }
}
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    int _ = 1;
    // cin >> _;
    while (_--)
        solve();
    return 0;
}
```

8.2. 树上启发式合并(dsu on tree)

https://atcoder.jp/contests/abc340/tasks/abc340_g

给一张图,求满足条件的子图 T 的数量

要求:

1. T 是一棵树

2. T 的所有叶子节点(度数为 1)颜色相同

```
void solve()
{
    ll ans = 0;
    int n;
    cin >> n;
    vector<int> a(n + 1);
    vector<vector<int>> e(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        e[u].push_back(v), e[v].push_back(u);
    }
    auto dfs = [&](auto dfs, int u, int fa) -> map<ll, ll>
    {
        map<ll, ll> fmp;
        for (auto to : e[u])
            if (to != fa)
            {
                auto smp = dfs(dfs, to, u);
                ans = (ans + smp[a[u]]) % mod; // u 与 to 组成树
                if (fmp.size() < smp.size())
                    swap(fmp, smp);
                for (auto [col, sz] : smp)
                {
                    ans = (ans + fmp[col] * sz % mod) % mod;
                    fmp[col] = ((fmp[col] + 1) * (sz + 1) % mod - 1 + mod) % mod;
                }
            }
        fmp[a[u]] = (fmp[a[u]] + 1) % mod;
        ans = (ans + 1) % mod; // u 自己作为树
        return fmp;
    };
    dfs(dfs, 1, 0);
```

```
    cout << ans << endl;
}
```

9. 可持久化数据结构

9.1. 可持久化线段树

每次单点操作,节点数量最多增加$\log(n)$

https://www.luogu.com.cn/problem/P3919

```
#include <bits/stdc++.h>
#define endl '\n'
#define x first
#define y second
#define ls(x) (a[x].l)
#define rs(x) (a[x].r)
#define sum(x) a[x].sum
using namespace std;
typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef double db;
const ll mod = 1e9 + 7;
const int N = 7e7 + 10, M = 25;
random_device rd;
mt19937_64 gen(rd());
struct node
{
    ll sum, l, r;
};
struct segtree
{
    int n;
    vector<node> a;
    segtree(int _n) : n(1) { a.reserve(_n); }
    // segtree(int _n) : n(1), a(n) {}//会re
    void update(int id)
    {
        sum(id) = sum(ls(id)) + sum(rs(id));
    }
```

```
    void build(int id, int l, int r, vector<int> &arr)
    {
        if (l == r)
            sum(id) = arr[l];
        else
        {
            ls(id) = ++n, rs(id) = ++n;
            int mid = (l + r) / 2;
            build(ls(id), l, mid, arr);
            build(rs(id), mid + 1, r, arr);
            update(id);
        }
    }
    void change(int pos, int l, int r, int id, int nxt, int val)
    {
        if (l == r)
        {
            sum(nxt) = val;
            return;
        }
        ls(nxt) = ls(id), rs(nxt) = rs(id);
        int mid = (l + r) / 2;
        if (pos <= mid)
            ls(nxt) = ++n, change(pos, l, mid, ls(id), ls(nxt), val);
        else
            rs(nxt) = ++n, change(pos, mid + 1, r, rs(id), rs(nxt), val);
        update(nxt);
    }
    ll query(int id, int l, int r, int ql, int qr) // 区间查询
    {
        if (l == ql && r == qr)
            return sum(id);
        int mid = (l + r) / 2;
        if (qr <= mid)
            return query(ls(id), l, mid, ql, qr);
        else if (ql > mid)
            return query(rs(id), mid + 1, r, ql, qr);
```

```
        else
            return query(ls(id), l, mid, ql, mid) + query(rs(id), mid + 1, r, mid + 1, qr);
    }
};
void solve()
{
    int n, m;
    cin >> n >> m;
    vector<int> a(n + 1), root(m + 10);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    segtree seg(n * 4 + m * (__lg(n) + 3));
    seg.build(1, 1, n, a);
    root[0] = 1;
    int nxt, pos, val, opt;
    for (int i = 1; i <= m; i++)
    {
        cin >> nxt >> opt >> pos;
        if (opt == 1)
        {
            cin >> val;
            root[i] = ++seg.n;
            seg.change(pos, 1, n, root[nxt], root[i], val);
        }
        else
        {
            root[i] = root[nxt];
            cout << seg.query(root[i], 1, n, pos, pos) << endl;
        }
    }
}
int main()
{
    // cout << setprecision(5);
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    int _ = 1;
    // cin >> _;
```

```
    while (_--)
        solve();
    return 0;
}
```

如果需要区间修改,则可以使用标记永久化来节省空间

```
1
```

9.1.1. 单点修改,区间查询

```
// 单点修改,区间查询主席树
struct Node {
    int l, r, sum;
};

struct ZX_tree {
    int tot = 0;
    Node tr[N << 5];
    void insert(int &root, int last, int l, int r, int v) {
        root = ++tot;
        tr[root] = tr[last];
        tr[root].sum++;
        if (l == r)
            return;
        int mid = (l + r) >> 1;
        if (v <= mid)
            insert(tr[root].l, tr[last].l, l, mid, v);
        else
            insert(tr[root].r, tr[last].r, mid + 1, r, v);
    }
    int query(int L, int R, int ql, int qr, int l, int r) {
        if (ql <= l && r <= qr)
            return tr[L].sum - tr[R].sum;
        int mid = (l + r) >> 1, ans = 0;
        if (ql <= mid)
            ans = query(tr[L].l, tr[R].l, ql, qr, l, mid);
        if (mid < qr)
            ans += query(tr[L].r, tr[R].r, ql, qr, mid + 1, r);
        return ans;
```

```
    }
} ZX;
```

9.1.2. 区间修改,区间查询

https://acm.hdu.edu.cn/showproblem.php?pid=4348

区间[l,r]加 d 并更新版本

查询当前版本[l,r]区间的和

查询 t 版本的[l,r]的区间和

回溯到历史版本 t

```
#include <bits/stdc++.h>
using namespace std;
const int N = 1e5 + 10;
using ll = long long;
int a[N], rt[N];
// 区间加,区间查询主席树
struct zxtree {
    int lc[N << 5], rc[N << 5], rt[N], tot;
    ll sum[N << 5], lazy[N << 5];
    void clear() {
        tot = 0;
    }
    void build(int &now, int l, int r) {
        now = ++tot;
        lazy[now] = lc[now] = rc[now] = 0;
        if (l == r) {
            sum[now] = a[l];
            return;
        }
        int mid = (l + r) >> 1;
        build(lc[now], l, mid);
        build(rc[now], mid + 1, r);
        sum[now] = sum[lc[now]] + sum[rc[now]];
    }
    void modify(int &now, int pre, int L, int R, int l, int r, ll val) {
        now = ++tot;
```

```
        lc[now] = lc[pre], rc[now] = rc[pre], lazy[now] = lazy[pre];
        sum[now] = sum[pre] + (min(R, r) - max(L, l) + 1) * val;
        if (l <= L && R <= r) {
            lazy[now] += val;
            return;
        }
        int mid = (L + R) >> 1;
        if (l <= mid)
            modify(lc[now], lc[pre], L, mid, l, r, val);
        if (r > mid)
            modify(rc[now], rc[pre], mid + 1, R, l, r, val);
    }
    ll query(int now, int L, int R, int l, int r, ll cc) {
        if (l <= L && R <= r)
            return sum[now] + cc * (R - L + 1);
        int mid = (L + R) >> 1;
        ll ans = 0;
        if (l <= mid)
            ans += query(lc[now], L, mid, l, r, cc + lazy[now]);
        if (r > mid)
            ans += query(rc[now], mid + 1, R, l, r, cc + lazy[now]);
        return ans;
    }
} zx;
void solve(int n, int q) {
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    zx.build(rt[0], 1, n);
    int now = 0;
    for (int i = 1; i <= q; i++) {
        string opt;
        int l, r, add, t;
        cin >> opt;
        if (opt[0] == 'Q') {
            cin >> l >> r;
            cout << zx.query(rt[now], 1, n, l, r, 0) << endl;
        } else if (opt[0] == 'C') {
```

```
            cin >> l >> r >> add;
            now++;
            zx.modify(rt[now], rt[now - 1], 1, n, l, r, add);
        } else if (opt[0] == 'H') {
            cin >> l >> r >> t;
            cout << zx.query(rt[t], 1, n, l, r, 0) << endl;
        } else {
            cin >> now;
        }
    }
}
int main() {
#ifndef ONLINE_JUDGE
    freopen("in.txt", "r", stdin);
    freopen("out.txt", "w", stdout);
#endif
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    int _ = 1;
    int n, q;
    while (cin >> n >> q)
        solve(n, q);
    return 0;
}
```

9.1.3. 应用

静态查询区间第 k 小

```
#include <bits/stdc++.h>
#define endl '\n'
#define x first
#define y second
#define ls(x) (a[x].l)
#define rs(x) (a[x].r)
#define sum(x) a[x].sum
using namespace std;
typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
```

```
typedef double db;
const ll mod = 1e9 + 7;
const int N = 7e7 + 10, M = 25;
random_device rd;
mt19937_64 gen(rd());
struct node
{
    ll sum, l, r;
};
struct segtree
{
    int n;
    vector<node> a;
    segtree(int _n) : n(1) { a.reserve(_n); }
    // segtree(int _n) : n(1), a(n) {}//会re
    void update(int id)
    {
        sum(id) = sum(ls(id)) + sum(rs(id));
    }
    void build(int id, int l, int r)
    {
        if (l == r)
            sum(id) = 0;
        else
        {
            ls(id) = ++n, rs(id) = ++n;
            int mid = (l + r) / 2;
            build(ls(id), l, mid);
            build(rs(id), mid + 1, r);
            update(id);
        }
    }
    void change(int pos, int l, int r, int id, int nxt, int val)
    {
        if (l == r)
        {
            sum(nxt) = sum(id) + val;
            return;
        }
```

```
        ls(nxt) = ls(id), rs(nxt) = rs(id);
        int mid = (l + r) / 2;
        if (pos <= mid)
            ls(nxt) = ++n, change(pos, l, mid, ls(id), ls(nxt), val);
        else
            rs(nxt) = ++n, change(pos, mid + 1, r, rs(id), rs(nxt), val);
        update(nxt);
    }
    ll query(int id, int l, int r, int ql, int qr) //某个版本的区间查询
    {
        if (l == ql && r == qr)
            return sum(id);
        int mid = (l + r) / 2;
        if (qr <= mid)
            return query(ls(id), l, mid, ql, qr);
        else if (ql > mid)
            return query(rs(id), mid + 1, r, ql, qr);
        else
            return query(ls(id), l, mid, ql, mid) + query(rs(id), mid + 1, r, mid + 1, qr);
    }
    ll kth(int id, int nxt, int l, int r, int k)
    {
        if (l == r)
            return l;
        int res = sum(ls(nxt)) - sum(ls(id));
        int mid = l + r >> 1;
        if (res >= k)
            return kth(ls(id), ls(nxt), l, mid, k);
        else
            return kth(rs(id), rs(nxt), mid + 1, r, k - res);
    }
};
void solve()
{
    int n, m;
```

```
    cin >> n >> m;
    vector<int> a(n + 1), root(n + 1), b;
    for (int i = 1; i <= n; i++)
        cin >> a[i], b.push_back(a[i]);
    sort(b.begin(), b.end());
    b.erase(unique(b.begin(), b.end()), b.end());
    segtree seg(1e7);
    root[0] = 1;
    seg.build(root[0], 1, n);
    for (int i = 1; i <= n; i++)
    {
        root[i] = ++seg.n;
        int pos = lower_bound(b.begin(), b.end(), a[i]) - b.begin() + 1;
        seg.change(pos, 1, n, root[i - 1], root[i], 1);
    }
    int l, r, k;
    for (int i = 1; i <= m; i++)
    {
        cin >> l >> r >> k;
        int ans = seg.kth(root[l - 1], root[r], 1, n, k);
        cout << b[ans - 1] << endl;
    }
}
int main()
{
    // cout << setprecision(5);
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    int _ = 1;
    // cin >> _;
    while (_--)
        solve();
    return 0;
}
```

查询区间小于等于 k 的元素的和

https://atcoder.jp/contests/abc339/tasks/abc339_g

```cpp
#include <bits/stdc++.h>
#define endl '\n'
#define x first
#define y second
#define ls(x) (a[x].l)
#define rs(x) (a[x].r)
#define sum(x) a[x].sum
using namespace std;
typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef double db;
const ll mod = 1e9 + 7;
const int N = 7e7 + 10, M = 25;
random_device rd;
mt19937_64 gen(rd());
struct node
{
    ll sum, l, r;
};
struct segtree
{
    int n;
    vector<node> a;
    segtree(int _n) : n(1) { a.reserve(_n); }
    void update(int id)
    {
        sum(id) = sum(ls(id)) + sum(rs(id));
    }
    void build(int id, int l, int r)
    {
        if (l == r)
            sum(id) = 0;
        else
        {
            ls(id) = ++n, rs(id) = ++n;
            int mid = l + r >> 1;
            build(ls(id), l, mid);
            build(rs(id), mid + 1, r);
```

```
            update(id);
        }
    }
    void change(int pos, int l, int r, int id, int nxt, int val)
    {
        if (l == r)
        {
            sum(nxt) = sum(id) + val;
            return;
        }
        ls(nxt) = ls(id), rs(nxt) = rs(id);
        int mid = l + r >> 1;
        if (pos <= mid)
            ls(nxt) = ++n, change(pos, l, mid, ls(id), ls(nxt), val);
        else
            rs(nxt) = ++n, change(pos, mid + 1, r, rs(id), rs(nxt), val);
        update(nxt);
    }
    ll query(int id, int l, int r, int ql, int qr) // 查询某个版本的区间和
    {
        if (l == ql && r == qr)
            return sum(id);
        int mid = l + r >> 1;
        if (qr <= mid)
            return query(ls(id), l, mid, ql, qr);
        else if (ql > mid)
            return query(rs(id), mid + 1, r, ql, qr);
        else
            return query(ls(id), l, mid, ql, mid) + query(rs(id), mid + 1, r, mid + 1, qr);
    }
    ll query(int id, int nxt, int l, int r, int ql, int qr) // 在 id-nxt 版本之间查询值域在 ql-qr 的和
    {
        if (ql > qr)
            return 0;
```

```
        if (l == ql && r == qr)
            return sum(nxt) - sum(id);
        int mid = l + r >> 1;
        if (qr <= mid)
            return query(ls(id), ls(nxt), l, mid, ql, qr);
        else if (ql > mid)
            return query(rs(id), rs(nxt), mid + 1, r, ql, qr);
        else
            return query(ls(id), ls(nxt), l, mid, ql, mid) + query(rs(id), rs(nxt), mid + 1, r, mid + 1, qr);
    }
};

void solve()
{
    ll n, q, mx = 0;
    cin >> n;
    vector<ll> a(n + 1), b;
    for (int i = 1; i <= n; i++)
        cin >> a[i], b.push_back(a[i]), mx = max(mx, a[i]);

    sort(b.begin(), b.end());
    b.erase(unique(b.begin(), b.end()), b.end());
    cin >> q;
    vector<ll> root(n + 1);
    ll pre = 0;
    root[0] = 1;
    segtree seg(2e7);
    seg.build(root[0], 1, n);
    for (int i = 1; i <= n; i++)
    {
        root[i] = ++seg.n;
        int pos = (lower_bound(b.begin(), b.end(), a[i]) - b.begin()) + 1;
        seg.change(pos, 1, n, root[i - 1], root[i], a[i]);
    }
    for (int i = 1; i <= q; i++)
    {
        ll l, r, val;
```

```
        cin >> l >> r >> val;
        l ^= pre, r ^= pre, val ^= pre;
        int pos = (upper_bound(b.begin(), b.end(), val) - b.begin());
        pre = seg.query(root[l - 1], root[r], 1, n, 1, pos);
        cout << pre << endl;
    }
}
int main()
{
    // cout << setprecision(5);
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    int _ = 1;
    // cin >> _;
    while (_--)
        solve();
    return 0;
}
```